

Waste & Trash Detection with YOLOv8n

Line-by-line viva guide to *waste_detection_yolov8n.ipynb* (Case Study 6)

The project in one breath. We take a small dataset of rubbish photos that is already labelled in YOLO format and split into train / val / test. We fine-tune a COCO-pretrained YOLOv8-nano object detector on it, pick the best epoch using the validation set, measure it once on the unseen test set (precision, recall, mAP@0.5, mAP@0.5:0.95), break the errors down into correct / missed / wrong-class / false-alarm using the confusion matrix, and finally run the model on new photos as a demo.

How to read this guide

- Each grey box is a line (or a tight group of lines) from the notebook, exactly as written. The text under it explains what it does and *why*.
- Green boxes at the end of each section are the questions an examiner is most likely to ask about that cell, with short answers.
- Section 8 is a rapid-fire revision sheet; read it last, just before you go in.

Pipeline at a glance

Step	Notebook cell	What comes out
1 Setup	imports	Libraries loaded; GPU check
2 Check data	read classes.txt, count files, write trash.yaml	Class list, split sizes, YOLO data config
Class balance	Counter over train labels	Bar chart of objects per class
Label check	draw label boxes with PIL	3 images with ground-truth boxes
3 Train	YOLO('yolov8n.pt').train(...)	runs/detect/yolov8n/ with best.pt, last.pt, plots
Curves	show results.png, train_batch0.jpg	Loss / mAP curves; mosaic example
4 Test	best.val(split='test')	Precision, recall, mAP; per-class table
5 Errors	confusion matrix → DataFrame	correct / missed / wrong class / false alarm
6 Demo	best.predict(...)	New images with predicted boxes

0. The intro markdown cell and folder layout

The first cell is just text. It states the **goal**: find each waste item in a photo, draw a **bounding box** around it, and name its **category** (Bottle, Can, Cardboard, Cup, Plastic, ...). That is **object detection** = localisation (where) + classification (what). It differs from *image classification* (one label for the whole image) and from *segmentation* (a label for every pixel).

The folder layout matters because YOLO finds labels automatically: for an image at `data/train/images/abc.jpg` it looks for `data/train/labels/abc.txt`. It does this by swapping the folder name `images` for `labels` in the path. `yolov8n.pt` is the pretrained weights file. `classes.txt` lists one class name per line.

.venv kernel: a Python virtual environment, an isolated folder of installed packages, so the project's library versions do not clash with the rest of the system. **Shift + Enter** runs a cell and moves to the next one. Cells must run top to bottom because later cells use variables made earlier (`CLASSES`, `RUN_DIR`, `best`, `metrics` ...).

1. Setup: imports

```
from pathlib import Path
```

`Path` is an object-oriented file path. It lets us write `DATA_DIR / "train" / "images"` with the `/` operator, and gives methods like `.glob()`, `.read_text()`, `.stem`, `.suffix`. It works the same on Windows and Linux.

```
from collections import Counter
```

`Counter` is a dictionary that counts things. `counts["Can"] += 1` works even if "Can" was never added before, because missing keys start at 0. We use it for the class-balance chart.

```
import pandas as pd
```

pandas gives us the **DataFrame** (a table) and **Series** (one labelled column). We use it for the per-class results table, the error-analysis table, saving them as CSV, and the bar chart (`Series.plot` uses matplotlib underneath).

```
import torch
```

PyTorch is the deep-learning framework YOLOv8 is built on. Here we use it only to check whether an NVIDIA GPU (CUDA) is available. Ultralytics uses it internally for the actual training.

```
from PIL import Image, ImageDraw
```

Pillow (PIL) is the image library. `Image` opens, converts and resizes images. `ImageDraw` draws rectangles and text on them. We use it to check the labels and to show the predictions.

```
from IPython.display import Image as Show, display
```

Jupyter's own `Image` class displays an image file inside the notebook. It is renamed **Show** with `as` because the name `Image` is already taken by PIL on the line above; without the rename the second import would overwrite the first. `display()` renders an object (image, DataFrame) in the output area, even from inside a loop.

```
from ultralytics import YOLO
```

The Ultralytics package provides the `YOLO` class. One object handles `.train()`, `.val()`, `.predict()` and `.export()`.

```
print("GPU available:", torch.cuda.is_available())
```

Prints True if PyTorch can see a CUDA GPU, otherwise False. Later the notebook uses this to choose the number of epochs and the image size. Training on a CPU is much slower, often 10-50× slower.

Likely viva questions on this cell

Q. Why use pathlib instead of strings?

A. It builds paths safely across operating systems and has built-in file methods (glob, read_text, stem).

Q. Why is IPython's Image imported as Show?

A. To avoid a name clash with PIL's Image; both are used in the notebook.

Q. What is CUDA?

A. NVIDIA's platform for running computations on the GPU. Neural networks are mostly matrix multiplications, which a GPU runs in parallel.

2. Check the dataset

YOLO label format: one .txt file per image, one line per object:

```
class_id x_center y_center width height      e.g.  2 0.51 0.43 0.22 0.35
```

All four box numbers are **normalised**: divided by the image width or height, so they lie between 0 and 1. The example means class 2, box centre at 51% across and 43% down, box 22% of the image wide and 35% tall. Normalised coordinates stay correct when the image is resized.

`class_id` is the **0-based** line number in `classes.txt`: line 1 is class 0, and so on. An image with no objects has an empty (or missing) label file. That is a *background* image and teaches the model what not to detect.

Three splits: **train** is what the weights learn from. **val** is checked every epoch to choose the best epoch and trigger early stopping, so we indirectly tune on it. **test** is touched once at the end for an honest score on unseen data.

Cell: read classes, count files, write the YAML

```
DATA_DIR = Path("data")
```

A relative path to the data folder, relative to where the notebook is running. Written in CAPITALS by convention because it is a constant for the whole notebook.

```
CLASSES = [c.strip() for c in (DATA_DIR / "classes.txt").read_text().splitlines() if c.strip()]
```

A **list comprehension**, read from the inside out:

- `(DATA_DIR / "classes.txt").read_text()` reads the whole file as one string.
- `.splitlines()` splits it into a list, one item per line.
- `if c.strip()` skips blank lines. An empty string is *false*, so blank lines are dropped.
- `c.strip()` removes spaces and newline characters from each name.

The result is a list such as `['Bottle', 'Can', ...]`. Its **index = class_id**, so this list is how we turn numbers into names.

```
print(len(CLASSES), "classes:", CLASSES)
```

Shows how many classes there are (8 according to the markdown) and their names, as a quick sanity check.

```
for split in ["train", "val", "test"]:
    n_img = len(list((DATA_DIR / split / "images").glob("*.jpg")))
    n_lbl = len(list((DATA_DIR / split / "labels").glob("*.txt")))
    print(f"{split:5}: {n_img} images, {n_lbl} label files")
```

Loops over the three splits. `glob("*.*)` matches every file whose name contains a dot, i.e. every image whatever its extension. `glob("*.txt")` matches only label files. `glob` returns a *generator*, so it is wrapped in `list()` before `len()` can count it.

`{split:5}` in the f-string pads the name to 5 characters so the three lines line up. **Why check?** The two counts should match. A mismatch means some images have no label file (the model would treat them as background) or there are stray label files.

```
DATA_YAML = DATA_DIR / "trash.yaml"
```

The path of the config file we are about to create. YOLO reads its dataset settings from a YAML file.

```
DATA_YAML.write_text(f"path: {DATA_DIR.resolve().as_posix()}\n"
                    "train: train/images\nval: val/images\ntest: test/images\n"
                    f"names: {CLASSES}\n")
```

Writes a 5-line text file. Python joins the three string pieces into one because adjacent string literals are concatenated automatically.

- **path**: the dataset root. `.resolve()` makes it an **absolute** path, so YOLO finds it whatever its working folder is. `.as_posix()` uses forward slashes, which avoids problems with Windows back-slashes in YAML.

- **train/val/test**: image folders *relative to* path. YOLO finds the labels by swapping images→labels.

- **names**: the Python list is printed as `['Bottle', 'Can', ...]`, which is also a valid YAML list. The order sets which `class_id` means which name.

`\n` is a newline character.

```
print("\n" + DATA_YAML.read_text())
```

Reads the file back and prints it, so we can see exactly what YOLO will receive.

Likely viva questions on this cell

Q. What is the YAML file for?

A. It tells YOLO where the train/val/test images are and what the class names are. It is passed as `data=` to train and val.

Q. Why are YOLO coordinates normalised?

A. So labels are independent of image resolution; YOLO resizes every image to `imgsz`, and the labels still fit.

Q. What happens if an image has no label file?

A. YOLO treats it as a background image with no objects. That is fine if it is intentional, but a bug if the label went missing.

Cell: class balance

```
counts = Counter()
```

An empty counter to hold the number of objects per class name.

```
for label_file in (DATA_DIR / "train" / "labels").glob("*.txt"):
```

Visits every label file in the **training** split. Only train is used because that is what the model learns from.

```
    for line in label_file.read_text().splitlines():
        if line.strip():
```

Each line is one object. Blank lines, such as a trailing newline, are skipped.

```
        counts[CLASSES[int(line.split()[0])]] += 1
```

Read it from the inside out: `line.split()` splits on spaces, giving `['2','0.51',...]`. `[0]` takes the first item, the class id as text. `int(...)` converts it to the number 2. `CLASSES[2]` gives the class name. `+= 1` adds one to that class's count.

Note this counts **objects** (boxes), not images. One image can hold several objects.

```
pd.Series(counts).sort_values().plot.barh(title="Objects per class (train)", figsize=(7, 4));
```

Turns the Counter into a pandas Series (names become the index, counts the values), sorts it smallest to largest, and draws a **horizontal bar chart** 7×4 inches in size.

The trailing **semicolon** stops Jupyter from printing the text of the returned matplotlib object (e.g. `<Axes: ...>`); only the chart is shown.

Why it matters: this is **class imbalance**. A class with few examples gives the model little to learn from, so it usually scores lower. That is why results are also reported per class and not only as one average. Possible fixes: collect more images of the rare classes, oversample them, or use weighted loss.

Cell: draw the labels on 3 training images

```
for img_path in sorted((DATA_DIR / "train" / "images").glob("*.jpg"))[:3]:
```

`sorted` gives a fixed, reproducible order (glob order is not guaranteed). `[:3]` takes the first three images.

```
img = Image.open(img_path).convert("RGB")
```

Opens the image and converts it to 3-channel RGB. This handles PNGs with a transparency channel (RGBA) and greyscale images, so we can draw in colour.

```
draw, (W, H) = ImageDraw.Draw(img), img.size
```

Two assignments on one line using tuple unpacking. `draw` is a drawing tool attached to `img`. `img.size` is (width, height), unpacked into `W` and `H`.

```
for line in (DATA_DIR / "train" / "labels" / f"{img_path.stem}.txt").read_text().splitlines():
```

`img_path.stem` is the file name without its extension (abc.jpg gives abc). So this opens the matching label file abc.txt and loops over its objects.

```
c, xc, yc, bw, bh = map(float, line.split())
```

Splits the line into 5 strings, converts each to float with `map`, and unpacks them into class, x-centre, y-centre, box width and box height (all normalised).

```
x1, y1 = (xc - bw / 2) * W, (yc - bh / 2) * H
```

YOLO format → pixel corner. Top-left x = centre – half the width. Multiplying by `W` (or `H`) converts the fraction to pixels.

Worked example: `W=1000, xc=0.51, bw=0.22 → x1 = (0.51 – 0.11)×1000 = 400 px`.

```
draw.rectangle([x1, y1, x1 + bw * W, y1 + bh * H], outline="red", width=max(2, W // 200))
```

Draws the box from the top-left corner (`x1,y1`) to the bottom-right corner (`x1+box width in px, y1+box height in px`). The line thickness grows with image size (`W // 200`, integer division) but is never below 2 px, so it stays visible on big and small photos.

```
draw.text((x1, y1), CLASSES[int(c)], fill="red", font_size=max(12, W // 30))
```

Writes the class name at the box's top-left corner. `int(c)` is needed because `c` was converted to a float (`2.0` → `2`) and list indices must be integers. The font scales with image width, minimum 12. (`font_size=` needs Pillow 10.1 or newer.)

```
img.thumbnail((500, 500))
display(img)
```

`thumbnail` shrinks the image **in place** to fit inside 500×500 and **keeps the aspect ratio**; it never enlarges. It is drawn at full size first, then shrunk, so the boxes stay in the right place. `display` shows it; inside a loop a bare expression would not be shown.

Why do this? It is a sanity check on the *labels themselves*. If the boxes are shifted, the wrong size, or carry the wrong names, then the coordinate format or class order is wrong, and training on those labels would be wasted. "Garbage in, garbage out."

Likely viva questions on this cell

Q. Convert YOLO (`xc,yc,w,h`) to corner (`x1,y1,x2,y2`)?

A. $x1=(xc-w/2)\cdot W$, $y1=(yc-h/2)\cdot H$, $x2=(xc+w/2)\cdot W$, $y2=(yc+h/2)\cdot H$.

Q. Why `int(c)`?

A. `map(float,...)` made the class id 2.0, and a list index must be an integer.

Q. `thumbnail` vs `resize`?

A. `thumbnail` keeps the aspect ratio, only shrinks and changes the image in place; `resize` forces an exact size and returns a new image.

3. Train YOLOv8n

Theory from the markdown cell. Know this well, because it is where the conceptual questions come from.

- **One-stage detector:** a single forward pass predicts every box and class at once ("You Only Look Once"). A *two-stage* detector such as Faster R-CNN first proposes regions and then classifies each one. That is more accurate on small objects but slower.
- **Backbone** (CSPDarknet with C2f blocks in v8): a convolutional network that turns pixels into feature maps. Early layers find edges, middle layers textures, deep layers object parts.
- **Neck** (PAN-FPN): mixes deep and shallow features and outputs **3 scales**, at strides 8, 16 and 32. For a 640 image that is 80×80, 40×40 and 20×20 grids. The fine grid finds small objects (bottle caps), the coarse one large objects (cardboard).
- **Head: decoupled** (separate branches for class and box) and **anchor-free**. Each grid cell directly predicts the distance to the box's left, top, right and bottom edges, instead of adjusting pre-set anchor box shapes as YOLOv5 did.
- **NMS:** many neighbouring cells predict the same object. NMS sorts boxes by confidence, keeps the best one, and deletes others of the same class that overlap it above an IoU threshold (Ultralytics default 0.7).
- **n = nano:** about 3.2 M parameters and 8.7 GFLOPs. YOLOv8m has about 25.9 M parameters, roughly 8× more. Model sizes $n < s < m < l < x$: bigger is more accurate but slower.
- **Transfer learning:** the weights come pretrained on **COCO** (about 118k training images, 80 classes including bottle and cup). The general visual features are reused, and only the final class layer is replaced (80 → our class count). That is why a few hundred images are enough.
- **Loss** = box loss (**CIoU**, which rewards overlap, centre distance and aspect ratio) + class loss (**BCE**, binary cross-entropy for each class) + **DFL** (Distribution Focal Loss, which models each box edge as a probability distribution, giving sharper edges). The optimiser changes the weights to reduce this total.

- **Augmentation** (on by default): **mosaic** (4 images tiled into 1, which gives varied scales, contexts and partly hidden objects), horizontal flip (probability 0.5), HSV colour jitter, random scale and translate. Mosaic is switched off for the last 10 epochs (*close_mosaic=10*) so the model finishes on realistic images. Augmentation is applied to **train only**, never to val or test.

Cell: training

```
GPU = torch.cuda.is_available()
```

True/False, stored so we can pick settings based on the hardware.

```
EPOCHS = 50 if GPU else 20
```

A **conditional expression** (Python's ternary): 50 epochs on a GPU, 20 on a CPU, to keep CPU training time reasonable. An epoch is one full pass over all training images.

```
IMGSZ = 640 if GPU else 416
```

Images are resized (letterboxed) to this square size before entering the network. 640 is the YOLOv8 default. 416 is roughly 2.4× fewer pixels, so faster on a CPU, but small objects lose detail. Both are **multiples of 32**, the network's largest stride, so the feature grids divide evenly.

```
model = YOLO("yolov8n.pt") # COCO-pretrained weights
```

Builds the YOLOv8n architecture and loads the COCO-pretrained weights from the .pt file (a PyTorch checkpoint). This is the transfer-learning starting point. Loading 'yolov8n.yaml' instead would start from random weights.

```
model.train(
    data=str(DATA_YAML),
```

Starts fine-tuning. **data** is our YAML (converted to a string). Ultralytics reads the class count from it and automatically rebuilds the head for our classes.

```
    epochs=EPOCHS,
    imgsz=IMGSZ,
```

The upper limit on training passes and the input resolution chosen above.

```
    batch=16,
```

16 images per forward/backward pass. The weights update once per batch, so iterations per epoch = $\text{ceil}(\text{train images} / 16)$. A bigger batch gives more stable gradients but needs more memory.

```
    patience=15, # early stopping
```

Early stopping: if the validation *fitness* has not improved for 15 epochs in a row, training stops before reaching EPOCHS. It saves time and limits overfitting.

Fitness in Ultralytics = $0.1 \times \text{mAP}@0.5 + 0.9 \times \text{mAP}@0.5:0.95$, so it is mostly box-quality-weighted mAP. The epoch with the best fitness is saved as **best.pt**; the final epoch as **last.pt**.

Note: with only 20 CPU epochs, patience=15 rarely triggers.

```
    seed=42, # reproducible results
```

Fixes the random number generators (weight initialisation of the new head, shuffling, augmentation), so running it again gives the same or very similar results. 42 is just a conventional choice.

```
    name="yolov8n", exist_ok=True, # results go to runs/detect/yolov8n
)
```

Output folder name inside runs/detect/. `exist_ok=True` reuses and overwrites that folder when re-running instead of creating yolov8n2, yolov8n3 ..., so later cells always find the files at the same path.

```
RUN_DIR = Path(model.trainer.save_dir)      # folder where YOLO saved weights, plots and results.csv
```

After training, `model.trainer` is the trainer object and `save_dir` is the folder it actually wrote to. We store the path rather than hard-coding it. It contains: weights/best.pt, weights/last.pt, results.csv, results.png, args.yaml, confusion matrix, PR/F1 curves, and train_batch*.jpg.

```
print("Saved to:", RUN_DIR)
```

Confirms the location.

Default settings not written in the code (good to know if asked): optimizer='auto' (AdamW for short runs, SGD for long ones), lr0=0.01 initial learning rate with warm-up (3 epochs) then decay, weight_decay=0.0005, pretrained backbone, AMP mixed precision on a GPU, and validation after every epoch.

Likely viva questions on this cell

Q. What is fine-tuning / transfer learning?

A. Start from weights learned on a big dataset (COCO) and continue training on ours. The early layers already know generic features, so we need less data and time.

Q. What is an epoch vs a batch vs an iteration?

A. Epoch = one pass over all training images. Batch = 16 images processed together. Iteration = one weight update (one batch).

Q. What is early stopping?

A. Stop when validation fitness hasn't improved for patience=15 epochs; keep the best epoch as best.pt.

Q. best.pt vs last.pt?

A. best = the epoch with the highest validation fitness; last = the final epoch (useful for resuming training).

Q. Why is the image size smaller on a CPU?

A. Compute grows with the number of pixels. 416^2 is about 42% of 640^2 , so it is faster, at the cost of small-object accuracy.

Q. Why anchor-free?

A. No anchor sizes to tune for the dataset, fewer predictions, simpler post-processing, and it generalises better to odd shapes.

Cell: training curves

```
display>Show(filename=str(RUN_DIR / "results.png"), width=900))
```

Shows the grid of plots YOLO saved from results.csv, with one point per epoch: train/box_loss, train/cls_loss, train/dfl_loss, the same three for val, and metrics/precision, recall, mAP50, mAP50-95.

Healthy: all losses fall, metrics rise, then both level off. **Overfitting:** training loss keeps falling while validation loss rises, meaning the model memorises the training images. **Underfitting:** both losses still high or still falling steeply at the end, so it needs more epochs.

```
display>Show(filename=str(RUN_DIR / "train_batch0.jpg"), width=600)) # a training batch after mosaic augmentation
```

A picture of the first training batch as the model actually saw it: mosaic tiles, flips and colour shifts, with ground-truth boxes. It proves augmentation is on and the labels are still aligned after augmentation.

Likely viva questions on this cell

Q. How do you detect overfitting?

A. Validation loss goes up (or val mAP falls) while training loss keeps going down.

Q. Why does mosaic help?

A. It gives varied backgrounds, scales and cut-off objects in one image, and puts more objects into each batch. That helps with the brief's 'different shapes, sizes and backgrounds' challenge.

4. Evaluate on the unseen TEST images

Metrics you must be able to define (with formulas):

- **IoU** = $\text{area}(\text{pred} \cap \text{true}) / \text{area}(\text{pred} \cup \text{true})$. 1 = perfect overlap, 0 = none. A prediction is a **True Positive** if $\text{IoU} \geq \text{threshold}$ *and* the class is right; otherwise it is a **False Positive**. A real object that no prediction matched is a **False Negative**.
- **Precision** = $\text{TP} / (\text{TP} + \text{FP})$: of what I detected, how much was right? (few false alarms)
- **Recall** = $\text{TP} / (\text{TP} + \text{FN})$: of what exists, how much did I find? (few misses)
- **F1** = $2\text{PR} / (\text{P} + \text{R})$. Ultralytics reports P and R at the confidence threshold that gives the best mean F1.
- **AP** (one class): sort detections by confidence, trace the precision-recall curve as the threshold drops, take the **area under it**. Ultralytics uses 101-point interpolation, as COCO does. AP summarises every threshold at once.
- **mAP@0.5** = mean of AP over all classes with IoU threshold 0.5. **mAP@0.5:0.95** = average of mAP at IoU 0.50, 0.55, ..., 0.95 (10 thresholds). It is stricter and rewards **tight** boxes; this is the main COCO metric.

```
best = YOLO(str(RUN_DIR / "weights" / "best.pt")) # the best epoch from training
```

Loads the saved best checkpoint into a new model object, so we evaluate the best epoch and not the last one.

```
metrics = best.val(data=str(DATA_YAML), split="test", imgsz=IMGSZ,
```

Runs validation, but `split="test"` makes it use the **test** images listed in the YAML instead of val. The same image size as training is used for a fair comparison.

`val()` internally predicts with a very low confidence threshold (0.001) so it can trace the *whole* PR curve, applies NMS, matches predictions to ground truth by IoU, and computes the metrics. It returns a **DetMetrics** object.

```
plots=True, # also builds the confusion matrix used in step 5
```

Saves `confusion_matrix.png` (raw and normalised), PR curve, F1 curve, P and R curves, and the `val_batchX_labels/pred.jpg` comparison images.

```
name="yolov8n_test", exist_ok=True)
```

Output goes to `runs/detect/yolov8n_test`, kept separate from the training run folder.

```
TEST_DIR = Path(metrics.save_dir)
```

The path of that output folder, stored for later cells.

```
print(f"\nPrecision : {metrics.box.mp:.3f}")
print(f"Recall : {metrics.box.mr:.3f}")
```

`metrics.box` holds the box-detection metrics. `mp` = **mean precision** over classes, `mr` = **mean recall**. `:.3f` formats the number to 3 decimal places.

```
print(f"mAP@0.5      : {metrics.box.map50:.3f}")
print(f"mAP@0.5:0.95 : {metrics.box.map:.3f}")
```

map50 = mAP at IoU 0.5. **map** = mAP averaged over IoU 0.5–0.95. The second is always $\leq$ the first because the stricter thresholds count fewer TPs.

Likely viva questions on this cell

Q. Why evaluate on test and not val?

A. Val was used to choose best.pt (and for early stopping), so its score is slightly optimistic. Test was never used for any decision, so it gives an honest score.

Q. Why is mAP@0.5:0.95 lower than mAP@0.5?

A. At IoU 0.9 or 0.95 a box must be almost pixel-perfect to count, so many detections that were correct at 0.5 become FPs.

Q. High precision, low recall means?

A. The model is cautious: what it finds is right, but it misses many objects. Lowering the confidence threshold would trade some precision for recall.

Q. Why does val use conf 0.001?

A. To collect predictions at every confidence level so the full precision-recall curve (and so AP) can be computed.

Cell: results per class

```
per_class = pd.DataFrame({
    "class":      [CLASSES[i] for i in metrics.ap_class_index],
```

Builds a table. **metrics.ap_class_index** is the list of class ids that actually appear in the test set; a class with no test objects would be left out. We convert each id to its name. Using this index keeps the names in the same order as the metric arrays below.

```
    "precision": metrics.box.p,
    "recall":    metrics.box.r,
```

Arrays with **one value per class** (same order as **ap_class_index**). **p** = precision, **r** = recall, at the best-F1 threshold.

```
    "AP@0.5":      metrics.box.ap50,
    "AP@0.5:0.95": metrics.box.ap,
}).round(3)
```

Per-class AP at IoU 0.5 and averaged over 0.5–0.95. The mean of the **ap50** column equals **map50** above. **.round(3)** rounds all numbers to 3 decimals.

```
per_class.to_csv("results_per_class.csv", index=False)
per_class
```

Saves the table to CSV for the report; **index=False** leaves out the 0,1,2... row numbers. A bare variable name on the last line of a cell makes Jupyter display it as a formatted table.

How to interpret: small objects (bottle caps, cigarette butts) and rare classes usually score lowest. Small objects cover few pixels, especially at **imgsz 416**, and fewer examples give fewer chances to learn. Classes that look alike (Plastic vs Bottle) lose points to confusion.

5. Error analysis: the confusion matrix

For detection, Ultralytics builds a matrix of size $(n+1) \times (n+1)$. The extra row and column are **background**. Only predictions with confidence ≥ 0.25 are counted, matched to ground truth at IoU ≥ 0.45 . **Rows = predicted class, columns = true class.**

	true Bottle	true Can	true background
pred Bottle	correct	wrong class	false alarm (FP)
pred Can	wrong class	correct	false alarm (FP)
pred background	missed (FN)	missed (FN)	— (unused)

```
display(Show(filename=str(TEST_DIR / "confusion_matrix.png"), width=700))
```

Shows the matrix image YOLO saved during `best.val(plots=True)`.

```
cm = metrics.confusion_matrix.matrix          # [predicted, true]; last row/column = background
```

The raw counts as a NumPy array of shape $(n+1, n+1)$. `cm[i, j]` = how many times the model predicted class *i* where the truth was class *j*. Index *n* (the last one) = background.

```
n = len(CLASSES)
```

The number of real classes, i.e. the index of the background row and column.

```
errors = pd.DataFrame({
```

Builds one row per class with four error types:

```
"correct":      cm.diagonal()[ :n],
```

The diagonal `cm[i,i]` = predicted class *i* when the truth was class *i*, i.e. **true positives**. `[ :n]` drops the last diagonal cell (background, background), which means nothing.

```
"missed":      cm[n, :n],
```

The **background row**: the model predicted 'nothing' but a real object of class *j* was there. These are **false negatives**, counted under the true class.

```
"wrong class":  cm[:n, :n].sum(axis=0) - cm.diagonal()[ :n],
```

`cm[:n, :n]` is the class-only block. `.sum(axis=0)` sums **down each column**: every detection that matched a true object of class *j*, whatever class it was given. Subtracting the correct ones leaves true objects of class *j* that were **found but given the wrong name**, e.g. a Plastic called Bottle. Check: correct + wrong class + missed = the total number of true objects of that class.

```
"false alarm":  cm[:n, n],
```

The **background column**: the model predicted class *i* where there was no real object. These are **false positives**, counted under the *predicted* class (note this asymmetry).

```
}, index=CLASSES).astype(int)
```

Rows are labelled with the class names. `astype(int)` changes the counts from float (NumPy's default) to whole numbers.

```
errors.loc["TOTAL"] = errors.sum()
```

`errors.sum()` adds up each column. `.loc["TOTAL"]` = appends that as a new row called TOTAL, giving the overall picture: is the model mostly missing things, mislabelling, or hallucinating?

```
errors.to_csv("error_analysis.csv")
errors
```

Saves the table (this time keeping the class-name index) and displays it.

Cell: see the errors on the images

```
for labels_img in sorted(TEST_DIR.glob("val_batch*_labels.jpg")):
```

During val, YOLO saves mosaics of the first few batches (up to 3): **_labels** = ground-truth boxes, **_pred** = model predictions. The file names say 'val' even though we ran on test; that is just YOLO's naming.

```
print(labels_img.name, " vs ", labels_img.name.replace("labels", "pred"))
```

Prints which pair is shown. **str.replace** builds the matching prediction file name.

```
display>Show(filename=str(labels_img), width=520), Show(filename=str(labels_img).replace("labels", "pred"), width=520))
```

Shows truth and prediction one after the other. Compare them: a box in labels with none in pred = **missed**; an extra box in pred = **false alarm**; same box but a different name = **wrong class**. Relate these to the brief: overlapping objects are often merged by NMS or missed, small items are missed, and busy backgrounds cause false alarms.

Likely viva questions on this cell

Q. Rows or columns: which is the truth?

A. In Ultralytics, rows = predicted and columns = true, with background as the last row and column.

Q. What is an FP vs an FN in detection?

A. FP = a box where there is no object, or with IoU below the threshold. FN = a real object with no matching box.

Q. Why can NMS cause misses with overlapping objects?

A. Two real objects of the same class that overlap strongly can have IoU above the NMS threshold, so one of the two boxes gets deleted.

Q. Which confidence does the matrix use?

A. 0.25, not 0.001, so it reflects what a deployed model at the default threshold would actually output.

6. Try the model on new images

```
MY_IMAGES = Path("my_test_images")
MY_IMAGES.mkdir(exist_ok=True)
```

Creates the folder if it does not exist; **exist_ok=True** means no error if it already does.

```
files = sorted(p for p in MY_IMAGES.iterdir() if p.suffix.lower() in {".jpg", ".jpeg", ".png"}) \
```

iterdir() lists everything in the folder. The generator keeps only files whose extension, lower-cased so .JPG also matches, is in the set. Set lookup is fast. The **** at the end continues the statement on the next line.

```
or sorted((DATA_DIR / "test" / "images").glob("*..*"))[:3]
```

The or fallback: an empty list is *falsy*. If the folder has no photos, **A or B** returns B, the first 3 test images, so the demo cell never fails.

```
for r in best.predict(source=[str(f) for f in files], conf=0.25, imgsz=IMGSZ, verbose=False):
```

Runs inference. **source** = a list of file paths (strings). **conf=0.25**: boxes with confidence below 25% are thrown away, then NMS runs. **verbose=False** stops per-image log lines. It returns a list of **Results** objects, one per image; **r** is one of them.

Raising conf gives fewer false alarms but more misses (precision up, recall down); lowering it does the opposite.

```
out = Image.fromarray(r.plot()[ :, :, ::-1])          # draw boxes; BGR -> RGB
```

`r.plot()` returns the image as a NumPy array with boxes, labels and confidences drawn on it, in **BGR** channel order (OpenCV's convention).

`[ :, :, ::-1]` keeps all rows and columns and **reverses the channel axis**, turning BGR into RGB. Without it, reds and blues would be swapped. `Image.fromarray` turns the array into a PIL image.

```
out.thumbnail((700, 700))
```

Shrinks it to fit within 700×700 for display, keeping the aspect ratio.

```
print(Path(r.path).name, "->", len(r.bboxes), "objects")
```

`r.path` is the source image path; `.name` is its file name. `r.bboxes` holds the detections (.xyxy coordinates, .conf, .cls), and `len` counts them.

```
display(out)
```

Shows the annotated image.

7. The conclusion cell

This cell is a template for the report. What to say for each point:

- **Overall:** quote test precision, recall, mAP@0.5 and mAP@0.5:0.95 from Section 4.
- **Best and worst classes:** explain using object size, number of training examples (the class-balance chart), and look-alike classes (off-diagonal cells of the confusion matrix).
- **Error types:** from the TOTAL row, which is largest: missed, wrong class or false alarm? Link it to overlapping objects, varied shapes and sizes, and cluttered backgrounds.
- **Limitations:** small dataset (163 original photos). The ready-made split probably contains augmented copies of the same photo in both train and test, which is **data leakage** and makes the test score optimistic. It also used a CPU-reduced imgsz and epoch count.
- **Improvements:** more (and more varied) images; split by original photo *before* augmenting; more epochs or a larger model (s/m); a higher imgsz for small objects; tuning conf and NMS IoU; a class-balancing strategy.

8. Rapid-fire revision sheet

Object detection vs classification vs segmentation?	Detection = boxes + labels for every object; classification = one label per image; segmentation = a label for every pixel.
Why YOLOv8n?	Fastest and smallest (~3.2 M params), fine on a laptop CPU and fast enough for a real-time conveyor-belt camera; the trade-off is slightly lower accuracy.
One-stage vs two-stage?	One-stage (YOLO, SSD) predicts boxes in one pass: fast. Two-stage (Faster R-CNN) proposes regions then classifies them: slower, sometimes more accurate.
What does 'anchor-free' mean?	The head predicts distances to the box edges from each grid point instead of adjusting preset anchor box shapes.
What are the 3 detection scales for?	Strides 8/16/32 give fine to coarse grids for small, medium and large objects.
What is IoU?	Intersection ÷ union of two boxes; a detection is correct at $\text{IoU} \geq 0.5$ with the right class (for $\text{mAP}@0.5$).
What is NMS?	Removes duplicate overlapping boxes of the same class, keeping the most confident one.
Precision / Recall formulas?	$P = \text{TP}/(\text{TP}+\text{FP})$; $R = \text{TP}/(\text{TP}+\text{FN})$.
mAP@0.5 vs mAP@0.5:0.95?	The first checks 'found it roughly'; the second averages over 10 IoU thresholds, so it rewards tight boxes.
What is the confidence threshold?	The minimum score for a box to be kept. Higher → precision ↑, recall ↓.
Transfer learning?	Starting from COCO-pretrained weights and fine-tuning on our data; this needs far less data and time.
Epoch / batch / imgsz?	One pass over the data / images per weight update / input resolution.
What is early stopping (patience=15)?	Stop if validation fitness hasn't improved for 15 epochs; the best epoch is saved as best.pt.
What makes up the loss?	CloU box loss + BCE class loss + Distribution Focal Loss.
What is mosaic augmentation?	Stitches 4 training images into one; gives varied scales and contexts, and objects cut at edges.
Why train / val / test?	Train learns; val chooses the epoch and stops early (so it is tuned on); test is used once for an honest score.
What is data leakage here?	Augmented copies of one photo in both train and test, so the model has effectively seen the test images and the scores are inflated.
Overfitting signs and fixes?	Val loss rises while train loss falls. Fixes: more data, more augmentation, early stopping, a smaller model.
What is class imbalance and its effect?	Some classes have far fewer examples, so their AP is lower. Fixes: more data for them, oversampling, weighted loss.
Why is the YOLO label format normalised?	It is independent of image size, so it survives resizing to imgsz.

What does [:, :, ::-1] do?	Reverses the colour channels: BGR (OpenCV) → RGB (PIL).
What does exist_ok=True do?	Reuses the same output folder instead of creating yolov8n2, yolov8n3, ...
Why seed=42?	Reproducibility: same random init, shuffle and augmentation each run.
Why is imgsz a multiple of 32?	The network downsamples by up to 32×, so the grids must divide evenly.
How would you deploy it?	best.export(format='onnx') (or TensorRT / TFLite), run it on a camera over the conveyor belt, and map each detected class to a sorting bin or actuator.
How would you improve small-object detection?	Higher imgsz (640 or more), more small-object examples, a larger model, tiling or SAHI (slicing) at inference.

Tip for the viva: when asked about any line, answer in three parts: *what it does* → *why it is needed* → *what would go wrong without it*. For example: "[:, :, ::-1] flips BGR to RGB, because r.plot() comes from OpenCV; without it the colours would be swapped."